

08 — Understanding Python classes with an experiment record

The question

What is a class? How are a class, an object, and an instance related? What do `self`, `__init__`, attributes, and methods actually do in research code?

This chapter builds a small record for one experiment run. Its measurements are invented teaching data, not actual experimental results. Review [03 — Python data types](#) if lists and dictionaries are unfamiliar. [05 — Dunder methods](#) explains special methods after the class basics make sense.

Run [the complete example](#) with Python 3.9+; it needs no external packages. Read sections 1–6 first, then run the example and inspect it in Spyder. Inheritance near the end is optional.

1. Start with data and an operation

Suppose one run has a name and several measurements in milliseconds:

```
name = "run A"
measurements = [10.0, 12.0]
mean_value = sum(measurements) / len(measurements)
```

This is already useful code. With many runs, you may want each name and measurement list to stay together, along with operations for adding a measurement and calculating its mean. A class lets us define that kind of record and its operations once, then create several records.

Term	Meaning in our example
Class	ExperimentRun: a type we define for experiment records
Object	A value Python can work with; numbers, lists, and our records are all objects
Instance	One object of a particular class; <code>run_a</code> refers to an instance of <code>ExperimentRun</code>
Attribute	A named item reached with a dot, such as <code>run_a.name</code>
Method	An operation accessed through an object, such as <code>run_a.mean()</code>

“Object” and “instance” are not opposing categories: the same record is an object and an instance of `ExperimentRun`. The class itself is also an object. Here we focus on the records created by calling that class. Python’s [classes tutorial](#)^[1] introduces these relationships.

2. Define a complete small class

```
class ExperimentRun:
    unit = "ms"

    def __init__(self, name, measurements):
        self.name = name
        self.measurements = list(measurements)

    def add_measurement(self, value):
        self.measurements.append(value)

    def mean(self):
        if not self.measurements:
            raise ValueError("Cannot calculate a mean without measurements.")
        return sum(self.measurements) / len(self.measurements)
```

Read it from the outside inward:

1. `class ExperimentRun:` defines our new type. Class names conventionally use capitalized words; this is a naming convention, not a requirement for execution.
2. The indented `def` blocks define methods. Their bodies are indented one level further, just like an ordinary function body.
3. `__init__` sets the starting state: a name and a measurement list. `self.name` and `self.measurements` are instance attributes.
4. `add_measurement` changes that instance's list. It has no explicit return, so its result is `None`.
5. `mean` returns a number without changing the list. An empty list raises a clear `ValueError`; its mean is not silently treated as zero.

The input contract for this teaching class is a string name and a flat sequence of numeric measurements in milliseconds. It does not validate every input or convert units. `unit = "ms"` is a label, not an instruction that performs numerical conversion.

3. Create an instance: what runs, and in which order?

```
run_a = ExperimentRun("run A", [10.0, 12.0])
```

For this ordinary class, read that line as follows:

1. Calling `ExperimentRun(...)` begins creating an instance. Python's inherited `__new__` supplies the new object; we do not need to implement it.
2. Python calls `__init__` with the new object as `self`, "run A" as name, and the list as measurements.
3. The assignments inside `__init__` store attributes on that object. The initialized object is returned by the class call and bound to the name `run_a`.

Strictly, `__init__` is an **initializer**, not the operation that creates the object. It must not return a non-None value; normally write no return in it. The [Python data model](#) ^[2] distinguishes it from `__new__`.

Defining the class does not run these method bodies. Calling the class creates a record; calling a method later performs that operation. `ExperimentRun` refers to the class, whereas `ExperimentRun(...)` calls it.

4. Understand self, parameters, and the dot

`self` means “the instance receiving this method call.” It is a conventional parameter name, not a keyword, global variable, or a second record. For our class, these calls have the same effect; execute only one if you want to add one value:

```
run_a.add_measurement(14.0)          # usual form
# ExperimentRun.add_measurement(run_a, 14.0) # equivalent explicit form
```

Python supplies `run_a` as `self` in the first form. You supply only `14.0`, which becomes `value`. When you call `run_b.add_measurement(...)`, the same method works with `run_b` as `self`.

Inside `__init__`, `name` is a local parameter; `self.name` is an attribute that remains on the instance after the method finishes. Writing only `name = name` would not store an instance attribute. The two names need not match: `self.name = label` works if the parameter is named `label`.

```
print(run_a.name)          # run A: read an attribute
run_a.name = "baseline"    # replace that attribute
print(run_a.measurements)  # [10.0, 12.0, 14.0]
print(run_a.mean())        # 12.0: call a method
calculate = run_a.mean     # store a bound method; no calculation yet
print(calculate())         # 12.0: now call it
```

The dot selects an attribute; parentheses call the selected method. `run_a.mean` and `run_a.mean()` are different expressions. Our attributes are public, so `run_a.measurements.append(16.0)` also changes the list. A class does not automatically prevent outside changes or ensure valid data.

5. Separate instances, copies, and aliases

Start this section with fresh records; it does not continue the optional changes above.

```
source = [10.0, 12.0]
run_a = ExperimentRun("run A", source)
run_b = ExperimentRun("run B", source)
run_a.add_measurement(14.0)
source.append(99.0)

print(run_a.measurements) # [10.0, 12.0, 14.0]
print(run_b.measurements) # [10.0, 12.0]
print(source)             # [10.0, 12.0, 99.0]
print(run_a is run_b)     # False
```

`list(measurements)` creates a new outer list on every initialization. Therefore the two records and `source` have three different lists. Replacing it with `self.measurements = measurements` would share the supplied list, so changing one record could unexpectedly affect the other record and the caller.

This is a **shallow copy**. It suffices for our flat list of immutable numbers; nested mutable objects would still be shared. Class instances are not automatically independent: the way their attributes are assigned determines what is shared.

```
alias = run_a
alias.name = "renamed A"
print(run_a.name)           # renamed A
print(alias is run_a)       # True
```

alias = run_a creates another reference to the same instance, not a new record. This is the same assignment rule you learned for lists in chapter 03.

6. Instance attributes and class attributes

Attribute	Where we assign it	Intended role
self.name	Inside __init__	This record's name
self.measurements	Inside __init__	This record's own list
unit	Directly in the class body	A common label, accessed as ExperimentRun.unit or run_a.unit

In this simple class, reading run_a.unit falls back to the class attribute if the instance has no attribute called unit. Assigning run_a.unit = "s" adds an instance attribute that hides the class label for that record. Assigning ExperimentRun.unit = "s" changes the class label seen by instances without their own override. Neither assignment converts measurements. See Python's [class and instance variables](#) ^[3].

A common mistake is putting a mutable list in the class body:

```
class BadRun:
    measurements = []      # one list shared through the class

first = BadRun()
second = BadRun()
first.measurements.append(10.0)
print(second.measurements) # [10.0]: surprising shared state
```

For one list per record, assign a fresh list to self.measurements in __init__. Also avoid a mutable default such as def __init__(self, measurements=[]): a default is evaluated once when the function is defined. If measurements are optional, use measurements=None and create a new list when it is None.

7. Run the example and check the result

From the repository root, run:

```
python examples/08_python_classes.py
```

Expected output:

```
run A: [10.0, 12.0, 14.0], mean = 12.0 ms
run B: [10.0, 12.0], mean = 11.0 ms
source: [10.0, 12.0, 99.0]
same instance: False
same measurement list: False
empty run: Cannot calculate a mean without measurements.
Verification passed.
```

Check the arithmetic yourself: $(10 + 12 + 14) / 3 = 12$ and $(10 + 12) / 2 = 11$. The script checks these values, list independence, and independence from the caller's list. It catches the intended empty-list error so execution can finish.

In Spyder, open the script and use these steps:

- 1. Run cells 1–2 in order. `run_a`, `run_b`, and `source` are top-level names so they are convenient to inspect.
- 2. In the console, evaluate `type(run_a)`, `isinstance(run_a, ExperimentRun)`, `vars(run_a)`, and `run_a.measurements`. Expect the class `ExperimentRun`, `True`, the instance's attribute dictionary, and `[10.0, 12.0, 14.0]`. A module prefix in the displayed type can vary with how you run the file.
- 3. Put a breakpoint on the `return sum(...)` line marked P1, then debug the whole file. On the first stop, `self.name` is "run A" and `self.measurements` has three numbers. Continue to the next call to see "run B" and two numbers.
- 4. Use the console if your Variable Explorer filters out custom objects. After editing and rerunning a class definition, recreate its instances too; existing instances still belong to the previous class object.

Spyder's [debugger guide](#)^[4] explains inspecting local variables at breakpoints. In this class, `vars(run_a)` shows instance attributes; `unit` is on the class and therefore does not appear there unless you assign an instance override.

Screenshot placeholder: Show Spyder paused at P1, with `self.name` and `self.measurements` visible in the console or Variable Explorer. Suggested file: `docs/images/08-class-self-inspection.png`. Replace this paragraph with the image when a screenshot is available.

8. When should you use a class?

Situation	A suitable starting point
Calculate a mean from a list once	A function, such as <code>mean(values)</code>
Group a few settings or exchange simple records	A dictionary, such as <code>{"name": "A", "measurements": [10, 12]}</code>
Several records each keep state and use the same related operations	A class such as <code>ExperimentRun</code>
Work with a numerical array or table	An existing NumPy or pandas type, when appropriate

A class is not required for research code. Choose it when it makes relationships clearer, not because a longer program must be object-oriented. Functions can still do the calculations used by methods; methods do not require different mathematics.

9. Recognize inheritance and composition

Inheritance defines a specialized kind of an existing class. This optional example adds an operator name while reusing the measurement methods:

```
class LabeledRun(ExperimentRun):
    def __init__(self, name, measurements, operator):
        super().__init__(name, measurements)
        self.operator = operator

labeled = LabeledRun("run C", [8.0, 10.0], "student")
print(labeled.mean())           # 9.0: inherited method
print(isinstance(labeled, ExperimentRun))  # True
```

For this single-inheritance example, `super().__init__(...)` initializes the base-class part of the same instance; it does not create another record. If you write a new initializer, call the base initializer when its setup is needed.

Composition means keeping another object as part of your object. `ExperimentRun` already uses composition: it contains a list in `self.measurements` and delegates storage to that list. A future study record could contain a list of `ExperimentRun` objects. Start with a small class; elaborate inheritance trees are unnecessary here.

10. Common mistakes and a short practice check

Symptom or code	Explanation and correction
<code>NameError</code> for <code>ExperimentRun</code>	Run the class definition before using it.
An instance method is defined without <code>self</code>	Python supplies the instance automatically; a missing parameter can cause an argument-count <code>TypeError</code> .
<code>name = name</code> in the initializer	This does not create an attribute; use <code>self.name = name</code> .
<code>run_a.mean</code> gives a method object	Add parentheses to calculate: <code>run_a.mean()</code> .
<code>run_a = ExperimentRun</code>	This binds a name to the class; call <code>ExperimentRun(...)</code> to create a record.
<code>__init__</code> returns a number or <code>self</code>	Remove that return value; the class call returns the instance.
One run unexpectedly changes another	Check for shared lists or aliases; use the intended copy boundary.
A typo such as <code>run_a.measurments</code>	Reading an absent attribute raises <code>AttributeError</code> ; check spelling and initialization.

1. Add `run_b.add_measurement(16.0)` to the section 5 example. Predict both means before running.
2. Add a method `count(self)` that returns `len(self.measurements)`. Explain why the call is `run_a.count()` with no explicit `self`.
3. Predict what `alias = run_a`; `alias.add_measurement(18.0)` changes. Does `run_b` change?
4. Explain why an empty run raises an error instead of returning `0.0`.

Check: exercise 1 gives a mean of 12.0 for A and $38 / 3$, approximately 12.6667, for B. Exercise 2 returns 3 for A at that point; Python supplies the instance. Exercise 3 changes A through its alias, leaving B unchanged. Exercise 4 has no observations to average; zero would incorrectly suggest an observed result.

Sources

Python's [classes tutorial](#) ^[1], [object initialization reference](#) ^[2], and [built-in inspection functions](#) ^[5] document the language behavior. Spyder's [debugger guide](#) ^[4] documents inspection while paused. The experiment record, numerical inputs, and exercises are teaching examples created for this handbook.

References

1. <https://docs.python.org/3/tutorial/classes.html>
2. https://docs.python.org/3/reference/datamodel.html#object.__init__
3. <https://docs.python.org/3/tutorial/classes.html#class-and-instance-variables>
4. <https://docs.spyder-ide.org/current/panes/debugging.html>
5. <https://docs.python.org/3/library/functions.html#vars>